

ENTERPRISE PLATFORM

High-Level Design Document

HLD — PART 1 OF 3

Executive Summary | Architecture Decision | System Topology

Document Control

Field	Detail
Document Title	High-Level Design — Enterprise IT Services Platform
Document Type	High-Level Design (HLD)
Version	1.0.0
Status	Draft — Under Review
Scope	Module 1: Authentication & Identity — Foundation for all future modules
Architecture Style	Modular Monolith → Microservices (phased evolution)
Target Scale	Phase 1: 10K users Phase 2: 500K users Phase 3: 5M+ users
Primary Stack	FastAPI + Next.js + PostgreSQL + Redis + Docker
Related Documents	LLD Part 1, LLD Part 2, Code Review Response, Security Document

1. Executive Summary

This document defines the High-Level Design for an enterprise IT services platform being built by a startup IT services company. The platform will eventually span multiple modules — authentication, user profiles, orders and payments, notifications, admin reporting, and a mobile API — but begins with Module 1: the Identity and Authentication system.

The central architectural challenge is a constraint every startup faces: the system must start at near-zero cost while being structurally capable of handling 500,000 concurrent users without requiring a fundamental rewrite. This document defines how that is achieved.

The Core Architectural Philosophy

Build a Modular Monolith today. Design every interface as if it will become a Microservice tomorrow. The code that runs in one process today should be extractable into an independent service with zero changes to its business logic. This is achieved through strict internal boundaries, async messaging from day one, and shared-nothing module design.

1.1 Business Goals This Architecture Must Serve

#	Business Goal	Architectural Implication
1	Launch quickly with minimal infrastructure cost	Single-server Docker Compose deployment is valid for Phase 1
2	Reuse Module 1 auth across all future projects	Auth is a standalone service from day one — no coupling to business logic
3	Add Modules 2-5 without redesigning the core	Shared API Gateway, shared event bus, shared database conventions
4	Handle traffic spikes without outages	Stateless services, Redis caching, horizontal scaling ready
5	Serve mobile app clients in future	REST API with JWT cookies + API versioning from day one
6	Multi-tenant capability for IT services business	Tenant isolation designed at schema level from Module 1

2. Architecture Decision — Why Modular Monolith First

Three architecture patterns were evaluated for this platform. The decision is documented here permanently so future team members understand why the platform is structured the way it is.

2.1 Options Evaluated

Pattern	Cost to Start	Scale Ceiling	Rewrite Risk
Pure Microservices	Very high — 8+ services, K8s, service mesh, observability stack	Unlimited	None — already distributed
Modular Monolith	Very low — single process, one DB, Docker Compose	High — 500K+ users before extraction needed	Low — modules are pre-designed for extraction
Traditional Monolith	Very low — simple to build	Low — hard to scale specific bottlenecks	High — no internal boundaries, extraction is painful
Serverless (Lambda/Cloud Run)	Near zero — pay per request	Very high	Medium — vendor lock-in risk

2.2 Decision: Modular Monolith with Extraction Triggers

Why Not Microservices from Day One?

Microservices solve a scale problem this platform does not yet have. They introduce operational complexity that would consume the entire engineering budget on infrastructure management rather than product development. The team would spend 80% of their time on Kubernetes, service mesh, distributed tracing, and inter-service authentication — and 20% building the actual product. That is the wrong ratio for a startup.

The Modular Monolith gives us:

- Single deployment — one Docker Compose command starts everything
- Single database transaction across modules — no distributed transaction complexity
- Zero network latency between modules — function calls, not HTTP
- Pre-designed internal boundaries — each module is already a candidate microservice
- Clear extraction triggers — defined below, not guessed

2.3 Extraction Triggers — When to Break a Module Out

A module graduates from the monolith to an independent microservice when ANY of these conditions are met:

#	Trigger Condition	Example
1	Module needs a different scaling profile than others	Notification module handles 10x more requests than auth
2	Module needs a different tech stack	Payments module needs Go for sub-millisecond processing
3	Module needs independent deployment cycle	Payment team deploys 5x per day, auth team deploys weekly
4	Module becomes a compliance boundary	Payments must be PCI-DSS isolated from other data
5	Module causes performance degradation for others	Heavy report queries starving auth database connections
6	Module is sold as a standalone product	Auth module licensed to external clients independently

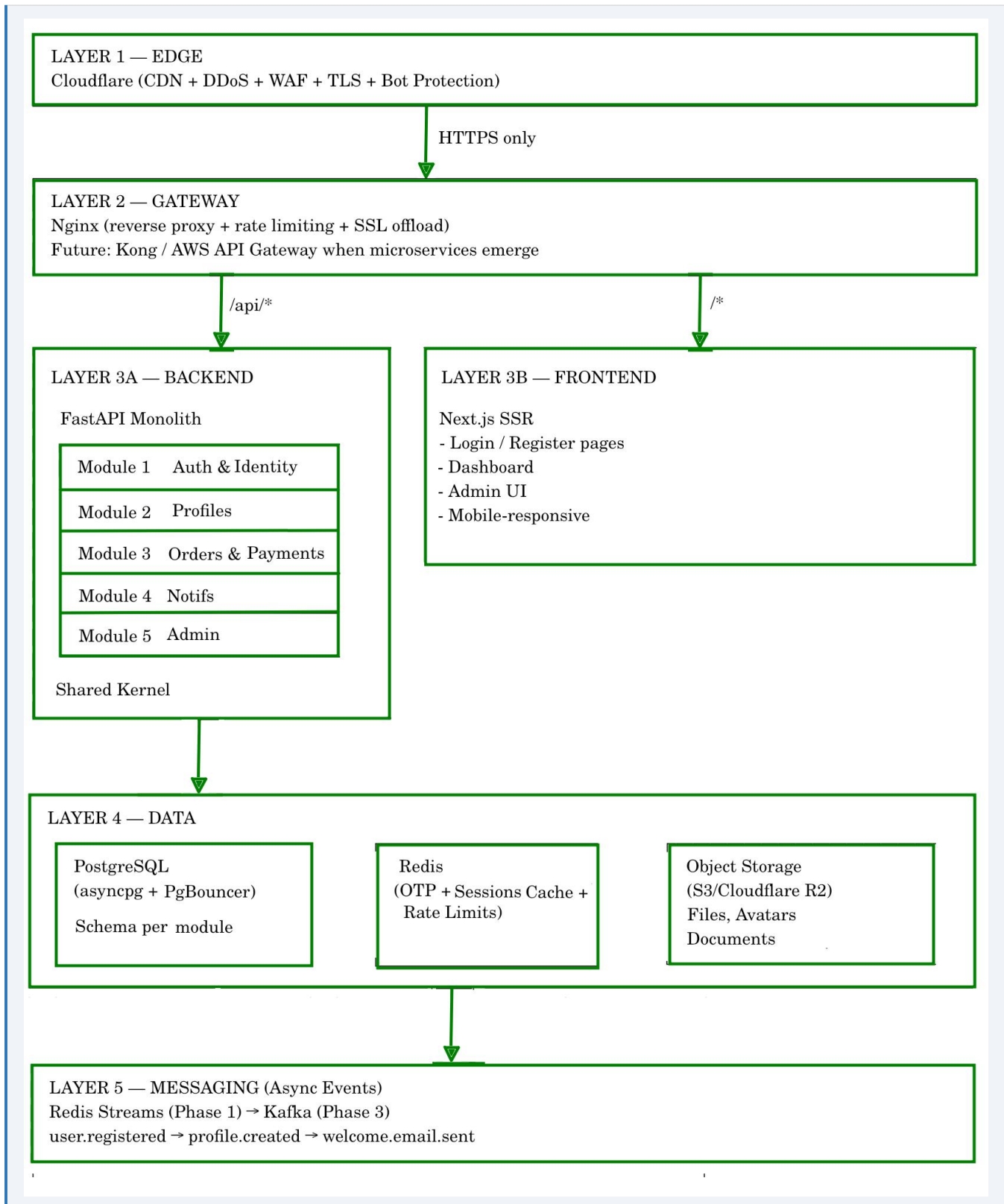
3. System Topology

The complete system is organised into five layers. Each layer has a single responsibility. Layers communicate only downward — never sideways, never upward except through defined response paths.

3.1 The Five Layers

L a y e r	Name	Responsibility
1	Edge Layer	CDN, DDoS protection, TLS termination, geographic routing. Cloudflare in Phase 1.
2	Gateway Layer	API Gateway, rate limiting, request routing, authentication token validation, API versioning.
3	Application Layer	All business modules run here. Phase 1: one process. Phase 2+: independent services.
4	Data Layer	PostgreSQL for persistent data, Redis for ephemeral data, S3-compatible storage for files.
5	Messaging Layer	Event bus for async communication between modules. Redis Streams in Phase 1, Kafka in Phase 3.

3.2 Complete Component Map



4. Complete Module Map — All Planned Modules

All five planned modules are defined here at the HLD level so the architecture can accommodate them from day one. Only Module 1 is implemented currently.

#	Module Name	Status	Core Responsibility
1	Identity & Auth	BUILT — Module 1	Registration, login (4 methods), OTP, JWT, sessions, password reset, audit log
2	User Profiles & Settings	PLANNED	Profile data, avatar, preferences, account settings, linked social accounts
3	Orders & Payments	PLANNED	Order lifecycle, payment processing, invoicing, refunds, payment gateway integration
4	Notifications	PLANNED	Email, SMS, push notifications, templates, delivery tracking, user preferences
5	Admin Dashboard	PLANNED	User management, system metrics, audit reports, role management, configuration
X	Mobile API	PLANNED	REST endpoints optimised for mobile clients, push token management, offline sync

4.1 Shared Kernel — What Every Module Shares

The Shared Kernel is a set of code that ALL modules use but NONE own. It contains infrastructure, not business logic. Adding a new module means importing the kernel — not copying code.

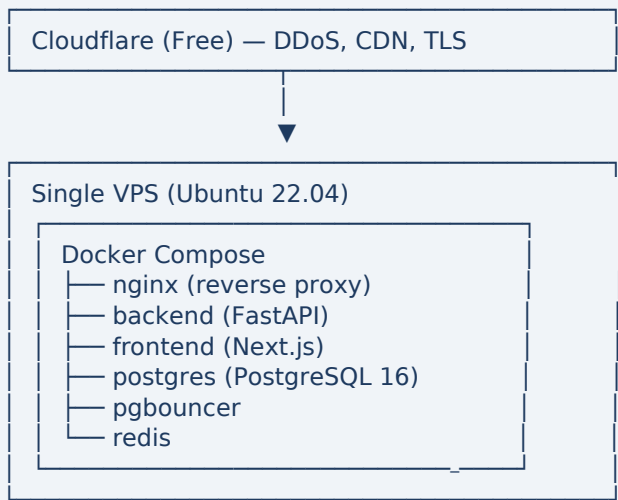
Kernel Component	What It Provides to All Modules
app/core/config.py	Single Settings class — all modules read environment variables from here
app/core/security.py	JWT decode, password hashing, token helpers — auth used by all modules
app/core/rate_limiter.py	Redis rate limiting — any module can rate-limit any endpoint
app/db/session.py	PostgreSQL connection pool — shared, not duplicated per module
app/db/redis.py	Redis connection pool — shared across all modules
app/middleware/	Request ID, security headers, error handler — applied to all modules
app/models/base.py	Base SQLAlchemy model with id, created_at, updated_at — all models inherit
app/events/bus.py	Event publisher — modules emit events, other modules subscribe

Kernel Component	What It Provides to All Modules
app/schemas/base.py	Base Pydantic schemas — standard error envelope, pagination, response wrapper

5. Deployment Topology — Phase by Phase

5.1 Phase 1 — MVP (Today, Near-Zero Cost)

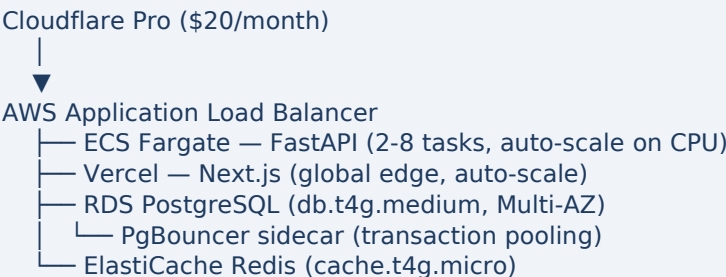
Single VPS — Hetzner CX31 or DigitalOcean Droplet
Cost: \$40-80/month



Handles: up to 10,000 daily active users comfortably
Scale up: resize the VPS, no code changes needed

5.2 Phase 2 — Growth (500K Users, Cloud Managed)

Cost: \$300-800/month



Handles: up to 500,000 daily active users
Code changes: ZERO — only infrastructure config changes

5.3 Phase 3 — Enterprise Scale (5M+ Users)

Cost: scales with traffic

Cloudflare Enterprise



AWS Global Accelerator → Multi-region ALBs

- EKS Kubernetes (HPA: 3-50 pods per service)
- Each module extracted as independent service
- API Gateway (Kong) — central routing, auth
- Aurora PostgreSQL Serverless v2
 - Writer endpoint (primary region)
 - Reader endpoints (3 AZs)
- ElastiCache Redis Cluster (3 shards)
- Kafka (async event bus replacing Redis Streams)
- Elasticsearch (audit log search)

Handles: 5 million+ daily active users
Code changes: ZERO — same Docker images, new deployment

6. API Design Strategy

6.1 URL Structure

All APIs follow this pattern:

```
/api/{version}/{module}/{resource}/{action}
```

Current endpoints (Module 1):

```
POST /api/v1/auth/register
POST /api/v1/auth/login/password
POST /api/v1/auth/login/otp/email/request
POST /api/v1/auth/login/otp/email/verify
POST /api/v1/auth/login/otp/sms/request
POST /api/v1/auth/login/otp/sms/verify
GET /api/v1/auth/login/google
GET /api/v1/auth/google/callback
POST /api/v1/auth/verify/email
POST /api/v1/auth/verify/phone
POST /api/v1/auth/password-reset/request
POST /api/v1/auth/password-reset/confirm
POST /api/v1/auth/refresh
POST /api/v1/auth/logout
GET /api/v1/auth/me
```

Future endpoints (planned structure):

GET	/api/v1/profiles/{user_id}	Module 2
PUT	/api/v1/profiles/{user_id}	Module 2
POST	/api/v1/orders	Module 3
GET	/api/v1/orders/{order_id}	Module 3
POST	/api/v1/notifications/send	Module 4
GET	/api/v1/admin/users	Module 5

6.2 Versioning Strategy

- All endpoints are versioned from day one — /api/v1/
- v1 is stable once a module goes to production — never break it
- New breaking changes go to v2 — v1 runs in parallel for 6 months minimum
- Version is in the URL, not a header — easier to test, cache, and route
- Mobile clients pin to a version — they cannot be forced to update immediately

6.3 Standard Response Envelope

Every API response — success or failure — follows the same envelope structure. All modules must use this. Consistency is critical for mobile and frontend clients.

Success response:

```
{
  "data": { ... },      // the actual payload
  "meta": {
    "request_id": "uuid", // for support tracing
    "timestamp": "ISO8601", // server time UTC
    "version": "1.0.0"    // API version
  }
}
```

Error response:

```
{
  "error": {
    "code": "RATE_LIMIT_EXCEEDED",
    "message": "Too many requests.",
    "field": null,
    "request_id": "uuid"
  }
}
```

Paginated list response:

```
{
  "data": [ ... ],
  "pagination": {
    "page": 1,
    "per_page": 20,
    "total": 1500,
    "total_pages": 75
  },
  "meta": { ... }
}
```

7. Non-Functional Requirements

Requirement	Phase 1 Target	Phase 2 Target	Phase 3 Target
API Response Time (p95)	< 500ms	< 200ms	< 100ms
API Response Time (p99)	< 1000ms	< 500ms	< 200ms
Availability (uptime)	99.5%	99.9%	99.99%
Concurrent Users	1,000	50,000	500,000
Daily Active Users	10,000	500,000	5,000,000
OTP Delivery (SMS)	< 30 seconds	< 10 seconds	< 5 seconds
Auth Token Validation	< 10ms	< 5ms	< 2ms
Database Query (simple)	< 50ms	< 20ms	< 10ms
Recovery Time Objective	4 hours	1 hour	15 minutes
Recovery Point Objective	24 hours	1 hour	5 minutes

8. Compliance and Regulatory Considerations

As an IT services company building for multiple clients, the platform must be designed with compliance boundaries in mind from the beginning.

Standard	What It Requires	How Architecture Addresses It
GDPR	Right to erasure, data portability, consent tracking	Soft delete pattern, audit logs, tenant isolation
PCI-DSS	Payment data isolation, encryption at rest	Module 3 designed as extractable service boundary
SOC 2	Audit trail, access controls, encryption	Immutable audit_logs table, RBAC in Module 5
India IT Act	Data localisation for Indian users	Region selection in cloud deployment config

HLD Part 1 Complete

Part 2 covers: Component Design in detail, Data Architecture, Multi-Tenancy strategy, Security Architecture, and the Event-Driven messaging design. Reply 'HLD Part 2' to continue.